Taller de Repaso e Integración: Recursividad, Búsqueda Secuencial y Punteros

Estudiante: Ian Quiroga **Curso:** 5.º año — 3.ª División (A-B) **Materia:** Laboratorio de Programación (LPR) **Institución:** E.E.S.T. N.º 1 «Eduardo Ader» — Vicente López **Docente:** Mansilla Muñoz, York Elías **Actividad:** 6 — Taller práctico de repaso y consolidación en C++ **Versión del documento:** 1.0.0 **Estado:** `v1.0.0 Baseline Aprobado` **Fecha:** 2026

Introducción

Esta actividad consolida de forma práctica, analítica y sin fricción teórica los **tres pilares fundamentales de bajo nivel** trabajados durante la primera mitad del ciclo lectivo. No se trata de teoría nueva: se trata de comprobar que los tres conceptos se entienden lo suficiente como para combinarlos en un mismo programa.

Los tres pilares son:

1. **Recursividad**: comprensión de la Pila de Llamadas (`Stack`), flujo de

llamadas autorreferenciales y la condición de corte obligatoria (Caso Base).

2. **Búsqueda secuencial**: recorrido indexado celda por celda en arreglos de

memoria contigua, optimizando el tiempo de ejecución de complejidad `O(n)` mediante interrupciones de ciclo (`break`).

3. **Punteros e intercambio físico**: modificación directa de variables

originales alojadas en la memoria mediante pasaje por dirección (`&`) y desreferenciación (`*`).

La conexión con el Proyecto Integrador Anual es directa: el control eficiente de la memoria de bajo nivel es la base para procesar datos de sensores y módulos de hardware en tiempo real.

Desarrollo

Cuestionario técnico de control

1. Caso Base

¿Qué error crítico ocurre en la memoria RAM si una función recursiva carece de un caso base de corte?

Cada llamada a `sumaRecursiva(n)` abre un **nuevo marco en el Stack** (la pila de memoria). Ese marco guarda las variables locales de esa invocación (`n`) y la dirección de retorno. Como el Stack tiene un tamaño finito y el sistema operativo le reserva solo una porción de la RAM, la pila no puede crecer indefinidamente.

Sin caso base, la función se llama a sí misma con `n - 1` para siempre: `5, 4, 3, 2, 1, 0, -1, -2, ...`. Como `n` es un entero que **no tiene límite inferior**, la cadena de llamadas no se detiene nunca, cada llamada reserva más memoria encima de la anterior y el Stack se desborda. Ese desbordamiento es el **Stack Overflow**.

Cuando el Stack se desborda, el sistema operativo detecta que el programa intenta escribir fuera de su espacio reservado y lo termina de inmediato con un error crítico (*segmentation fault* en Linux o *stack overflow* en Windows). No es un error controlado por el programa: es el sistema cortando el proceso antes de que pise la memoria de otro proceso.

En la implementación, el caso base `if (n <= 0) return 0;` es lo que corta la cadena: en el momento en que `n` llega a `0`, la función devuelve sin llamar a nadie más y el Stack empieza a vaciarse a medida que cada llamada retorna.

2. Eficiencia

¿Por qué la instrucción `break` optimiza el rendimiento del microprocesador al realizar una búsqueda en arreglos?

La búsqueda secuencial es un algoritmo de complejidad `O(n)`: recorre el arreglo índice por índice comparando cada celda con el valor buscado. En el peor caso recorre **las `n` posiciones** del arreglo, porque el valor buscado puede estar al final.

Cada vuelta del `for` implica:

• Cargar el valor de `vectorDatos[i]` desde la RAM.

• Compararlo con `numeroBuscar` en el procesador.

• Incrementar `i` y evaluar la condición del bucle.

Sin `break`, el procesador ejecuta esas operaciones para las `n` posiciones, aunque ya haya encontrado el valor en la posición 3. Es trabajo desperdiciado: el resultado ya se conoce y el programa sigue calculando. La CPU queda ocupada ejecutando comparaciones estériles, lo que además impide que el sistema atienda otras tareas.

Con `break`, en cuanto la comparación es verdadera, el `break` lanza un salto incondicional que aborta el `for` inmediatamente. El resultado visible en consola es el mismo, pero el procesador dejó de trabajar en el instante en que la búsqueda tuvo éxito.

La ganancia es cuantificable: si el valor está en la posición 3 de un arreglo de 10, el programa realiza **3 comparaciones en lugar de 10**. En el caso extremo — el valor no existe— el comportamiento es idéntico con o sin `break`, porque el bucle se agota de todas formas. Es decir, `break` **nunca empeora el caso peor y siempre mejora el caso promedio**, que es exactamente la propiedad que se busca en una optimización.

3. Direccionamiento

¿Cómo comprobar mediante el operador `&` que los elementos de un array ocupan celdas contiguas en la RAM?

Basta con imprimir las direcciones de dos elementos consecutivos y restarlas:

```
cout << &vectorDatos[i] << endl;      // por ejemplo 0x61fe1c
cout << &vectorDatos[i + 1] << endl;  // por ejemplo 0x61fe20
```

Un **arreglo contiguo** es un bloque de memoria donde los elementos se almacenan en posiciones consecutivas. La diferencia entre las dos direcciones da el tamaño en bytes de un elemento:

```
0x61fe20 - 0x61fe1c = 0x4 = 4 bytes
```

Como cada `int` ocupa 4 bytes en una arquitectura de 32 bits, el resultado confirma que no hay huecos entre los elementos: la memoria es **contigua**. Si el arreglo estuviera mal declarado o si los elementos se asignaran por separado, la diferencia sería mayor o irregular.

La comprobación completa consiste en imprimir todas las direcciones y verificar que la diferencia entre cada par consecutivo sea exactamente `sizeof(int)`:

```
sizeof(vectorDatos[0])   // devuelve 4
```

4. Variables Temporales

¿Qué función cumple la variable temporal en el algoritmo de intercambio de valores con punteros?

```
void intercambiarValores(int* ptrA, int* ptrB)
{
    int temporal = *ptrA;   // Guardamos A antes de sobrescribirlo
    *ptrA = *ptrB;          // A ahora vale lo que valía B
    *ptrB = temporal;      // B ahora vale lo que valía A
}
```

La variable `temporal` funciona como **contenedor de respaldo** durante el traslado del dato. Sin ella, la asignación sería:

```
*ptrA = *ptrB;   // A = B  (el valor original de A se destruye aquí)
*ptrB = *ptrA;   // B = A  -> pero A ya vale B, así que B = B. ¡Se pierde todo!
```

El valor original de `x` se habría perdido en la primera línea y no habría forma de recuperarlo en la segunda. `temporal` es el único lugar donde el valor antiguo sobrevive mientras se performs la escritura.

Además cumple una segunda función de rendimiento: es una variable **local** de la función, por lo que vive en el Stack y se destruye automáticamente al terminar la llamada, sin necesidad de memoria dinámica ni de mecanismos de liberación.

El hecho de que el intercambio funcione sobre las **variables originales** `x` e `y` (y no sobre copias) se debe a que la función recibe punteros: `*ptrA` y `*ptrB` no son copias de `x` e `y`, **son los mismos casilleros de RAM**.

5. Autoevaluación

De los tres retos, el más complejo de comprender a bajo nivel fue **el reto 1 (suma recursiva)**. La dificultad no está en escribir la función, sino en visualizar lo que ocurre en la memoria: entender que cada llamada reserva un marco nuevo en el Stack y que las llamadas se *apilan* unas encima de otras hasta que la última retorna y la pila se empieza a vaciar.

Se resolvió dibujando a mano la traza de la pila para `n = 5`, anotando en cada paso qué valor tiene `n` y qué hay en el marco de esa llamada:

```
[5] sumaRecursiva(5)
  ■■[4] sumaRecursiva(4)
      ■■[3] sumaRecursiva(3)
          ■■[2] sumaRecursiva(2)
              ■■[1] sumaRecursiva(1)
                  ■■[0] sumaRecursiva(0)  -> caso base, retorna 0
              ■■ retorna 1 + 0 = 1
          ■■ retorna 2 + 1 = 3
      ■■ retorna 3 + 3 = 6
  ■■ retorna 4 + 6 = 10
■■ retorna 5 + 10 = 15
```

Al verla por escrito, se entiende que la recursión no es «música», sino que es literalmente una pila: se acumulan marcos y luego se desarman de abajo hacia arriba. El **reto 3** (intercambio con punteros) fue más sencillo porque es la misma idea de la Actividad 4, solo que con dos variables

en lugar de una.

Estrategia anti-caídas: protección Dual-Remote (GitHub + GitLab)

Un Técnico en Programación no puede depender de un único servidor. Si GitHub cae o se pierde el acceso a la cuenta, el código no puede perderse. La solución implementada es el **Dual-Remote**: configurar Git para que cada `git push` suba automáticamente a GitHub (servidor principal) y a GitLab (servidor de emergencia).

La configuración se hace con tres comandos:

```
git remote set-url origin https://github.com/pachusUwu/repasogeneral.git
git remote set-url --add --push origin https://github.com/pachusUwu/repasogeneral.git
git remote set-url --add --push origin https://gitlab.com/TU-USUARIO/repasogeneral.git
```

El primer comando fija la **URL de lectura** (fetch). El segundo agrega GitHub como **primer destino de escritura** (push). El tercero agrega GitLab como **segundo destino de escritura**.

La verificación se hace con `git remote -v`, que debe mostrar **tres líneas**: una de fetch apuntando a GitHub, y dos de push (GitHub primero y GitLab segundo).

A partir de ese momento, `git push` sube a ambos servidores con un solo Enter. El proyecto en GitLab debe crearse **vacío** (sin marcar *"Initialize repository with a README"*), porque un `README` inicial genera un commit que después provoca un conflicto al intentarse fusionar el historial.

Tabla de control de revisiones (DocVer)

Versión Fecha Descripción Estado
`v1.0.0` 2026 Línea base del código provisto por el docente. Suite unificada de los 3 retos (recursividad, búsqueda secuencial e intercambio con punteros). **Baseline Aprobado**

Conclusión

Los tres pilares de bajo nivel resultaron ser mucho más dependientes entre sí de lo que parecían por separado. La recursividad y el intercambio con punteros comparten la misma idea de fondo: **trabajar sobre la ubicación de un dato, no sobre su copia**. En un caso la ubicación es un marco del Stack, en el otro es un casillero de la variable en la RAM. Entender que la referencia es el mecanismo común explica por qué las dos técnicas existen y cuándo conviene usar cada una.

La búsqueda secuencial, en cambio, plantea un problema de **eficiencia**, no de memoria: el dato está donde corresponde, la pregunta es cuánto trabajo hace el procesador para encontrarlo. `break` es la respuesta mínima y elegante.

La actividad también dejó una lección de/documentation que va más allá del C++: **un código sin historial de versiones es un código que todavía no existe como proyecto profesional**. Configurar el Dual-Remote es entender que la disponibilidad del código es una responsabilidad técnica, y que una copia de seguridad en otro servidor cuesta tres comandos.

Referencias

• cppreference.com. (2024). *Recursion*. C++ reference. <https://en.cppreference.com/w/cpp/language/function>

• Microsoft. (2024). *Stack overflow*. Microsoft Learn. <https://learn.microsoft.com/es-es/cpp/cpp-reference>

• GeeksforGeeks. (2024). *Linear Search*. <https://www.geeksforgeeks.org/linear-search/>

• Microsoft. (2024). *Arrays (C++)*. Microsoft Learn. <https://learn.microsoft.com/es-es/cpp/cpp/arrays-cpp>

• Git. (2024). *Git basics — Working with remotes*. Git Documentation. <https://git-scm.com/book/en/v2/Git-Basics-Working-with-Remotes>